

Ch# 20 String Matching :-

- $\Rightarrow$ "Text editing programs" frequently need to find all occurrences of a pattern in the text.
- $\Rightarrow$ Typically, the text is a document being edited, and the pattern searched for is a particular word supplied by the user.
- $\Rightarrow$ Efficient algorithm for this problem called string matching.
- $\Rightarrow$ Also called "String Searching algorithms".
- $\Rightarrow$ The string-matching problem is the problem of finding all valid shifts with which a given pattern P occurs in a given text T .
- $\Rightarrow$ valid shift: if P occurs with shift s in T , then s is called a valid shift.
- $\Rightarrow$ Otherwise it is called invalid shift.

Applications :-

- 1) Internet search engines use them to find web pages relevant to queries.
- 2) Search for particular patterns in DNA sequence.

Algorithms for String matching

These are different types of algorithms for string matching.

Naive

Rabin Karp

Finite automaton

Knuth morris-pratt

Naive String matching

⇒ Naive algorithm is exact string matching (means finding one or all exact occurrences of a pattern in a text) algorithm.

⇒ This algorithm is helpful for smaller texts.

⇒ it does not need any pre-processing phases.

⇒ we can find substring by checking once for the string.

⇒ It finds all valid shifts using a loop that checks the condition $P[1 \dots m] = T[s+1 \dots s+m]$ for each of the $n-m+1$ possible values of s .

Naive - STRING - MATCHER (T, P)

$n = T.length$

$m = P.length$

for $s = 0$ to $n - m$

if $P[1..m] == T[s+1..s+m]$

print "Pattern occurs with shift"

Time Complexity:

$\Rightarrow O((n-m+1)m)$

$\Rightarrow$ Preprocessing time is zero.

Note: The naive string-matcher is inefficient because it entirely ignores information gained about the text for one value of s when considering other values of s .

Dry Run:

T:

a	c	a	a	b	c
---	---	---	---	---	---

P:

a	a	b
---	---	---

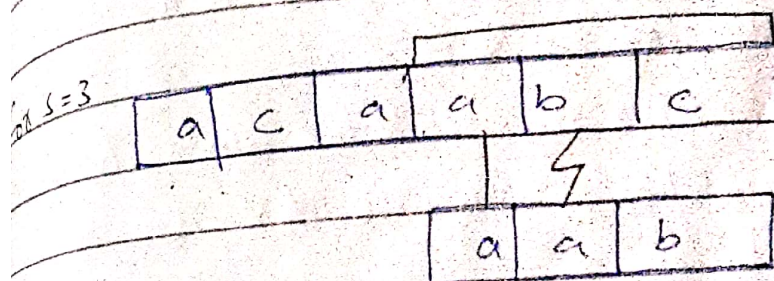
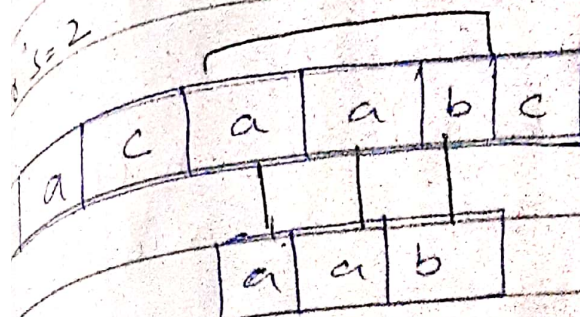
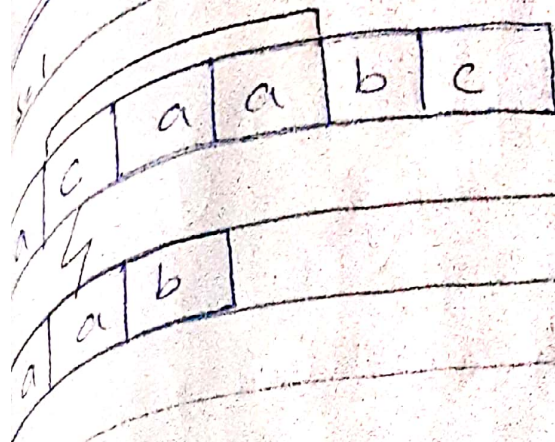
for $s = 0$

T:

a	c	a	a	b	c
---	---	---	---	---	---

P:

a	a	b
---	---	---



Pros:

- No pre-processing phase
- No extra operating space required
- Comparison of the patterns with the string can be done in any order.

Cons:

- Inefficient, because it has found a position, it does not use it again to find the other position, it does not use the information from the previous shift again.

Rabin-Karp Algorithm:-

$\Rightarrow$ It is used for matching pattern in the text through a hash function.

$\Rightarrow$ It first filters the characters that do not match and then performs the comparison.

$\Rightarrow$ Hash function is used to map a large input value to a smaller O/P value. Output value is called hash value.

Hit:- when hash value match it is called hit.

Valid hit:- if the pattern and hash value both matches then the hit is called valid hit.

Suspicious hit:- if the pattern does not match then the hit is called suspicious hit.

$\Rightarrow$ we choose such hash functions so that suspicious hit should be minimum.

Time Complexity:

Worst case : $O((n-m+1)m)$ Processing time:

Best case : $O(mn)$ - $H(m)$

Rabin-Karp-MATCHER (T, P, d, v)

$n = T.length$

$m = P.length$

$h = d^{m-1} \bmod v$

$P = 0$

$ts = 0$

for $i = 1$ to m

$P = (dP + P[i]) \bmod v$

$ts = (dts + T[i]) \bmod v$

for $s = 0$ to $n - m$

if $P == ts$

if $P[1..m] == T[s+1..s+m]$

print "patterns occur
with shift", s

if $s < n - m$

$ts+1 = (d(ts - T[s+1]h) +$

$T[s+m+1]) \bmod v$

te: if the value of t is negative then
add the value of v to t .

- $d =$ no of alphabets = 26 or 3

- $v =$ any random prime number.

T: a a a b a a b a a b

P: a a b

1 1 1 2 1 1 2 1 1 2

a a a b a a b a a b

0 1 3 4 1 3 0 1

$n=10, m=3, d=13, c=3$

$$f = d^m \text{ mod } c$$

$$f = d^m \text{ mod } 13 = 3^3 \text{ mod } 13 = 9$$

$f = 9$

for $i = 1$ to 3

$$i=1 \quad P = (3 \times 0 + P[1]) \text{ mod } 13 = 1 \text{ mod } 13 = 1$$

$$i=2 \quad P = (3 \times 1 + P[2]) \text{ mod } 13 = (3 + 1) \text{ mod } 13 = 4$$

$$i=3 \quad P = (3 \times 4 + P[3]) \text{ mod } 13 = (12 + 2) \text{ mod } 13 = 1$$

$P = 1$

$$i=1 \quad t_0 = (3 \times 0 + T[1]) \text{ mod } 13 = 1 \text{ mod } 13 = 1$$

$$i=2 \quad t_0 = (3 \times 1 + T[2]) \text{ mod } 13 = (3 + 1) \text{ mod } 13 = 4$$

$$i=3 \quad t_0 = (3 \times 4 + T[3]) \text{ mod } 13 = (12 + 1) \text{ mod } 13 = 0$$

for $s = 0$ to 7

check $t = 0$ false

$$t_1 = (3 \times 0 + T[4] \times 9) + t_0[4] \text{ mod } 13$$

$$= (3 \times 1 + 9) + 2 \text{ mod } 13$$

$$= (3 + 9 + 2) \text{ mod } 13 = 14 \text{ mod } 13 = 1$$

$t_1 = 1$

$S=1$

$$t_2 = (3(1 - (1)9) + 1) \bmod 13$$

$$= 3(1 - 9) + 1 \bmod 13 = -22 \bmod 13 = -9$$

$$t_2 = -9 + 13 = 4$$

$S=2$

$$t_3 = (3(4 - (1)9) + 1) \bmod 13$$

$$= 3(-5) + 1 \bmod 13 = -14 \bmod 13 = -1$$

$S=3$

$$t_4 = (3(1 - T[4]9) + T[7]) \bmod 13$$

$$= (3(1 - 18) + 2) \bmod 13 = (-34 + 2) \bmod 13$$

$$= -32 \bmod 13 = 1$$

Point :- $S = 1, 4, 7$

Knuth-Morris-Pratt Algorithm

- $\Rightarrow$ it uses concept of prefix and suffix
- $\Rightarrow$ Knuth Morris Pratt is linear time matching algorithm run in " $O(n)$ ".
- $\Rightarrow$ KMP use the property of degenerating.
- $\Rightarrow$ Degenerating property:
 - o Pattern having the same sub-patterns appearing more than once in the pattern.
- $\Rightarrow$ This algorithm was conceived in 1974 by Donald Knuth and Vaughan Pratt, and independently by James H. Morris.
- $\Rightarrow$ Knuth-Morris-Pratt algorithm compares the pattern to the text in left-to-right, but shifts the pattern more intelligently than the brute-force algorithm.

Components of KMP:

- i) The Prefix function π
 - o for a pattern encapsulates knowledge about how the pattern matches against shifts of itself.
 - o This information can be used to avoid useless shifts of the pattern P.
 - o In other words, it enables us avoiding backtracking on the pattern.

KMP Matcher

with text, "T", pattern P and preprocess
function π , finds the occurrence of P in
T and return the number of shifts
of "P" after which occurrence is found

KMP compares character by character
from left to right. But if a mismatch
occurs, it uses a preprocessed table
called "Prefix Table" to skip characters
comparison while matching.

PS table

we try to find longest proper prefix
KMP.

for this purpose we used a table in
which every character of the pattern
map to a value.

This table help us avoiding the
unnecessary old comparisons on finding the
partial matches.

Main Points:

Take 2 variables i and j $i = T[i]$, $j = P[j]$.

Computes i & j+1

- if match move i and j to right
- if mismatch then backtrack to previous

KMP-Matcher (T, P) — $O(m)$

$n = T \text{ length}$ — 1

$m = P \text{ length}$ — 1

$\pi = \text{Compute-Prefix-function}(P)$

$v = 0$ // number of characters matched

for $i = 1$ to n // scan the text from L-to-R

While $v > 0$ and $P[v+1] \neq T[i]$ — $m-1$

$v = \pi[v]$ // next character does not match

if $P[v+1] = T[i]$ — 1

$v = v + 1$ // next character matches

if $v == m$ // is all of P matched?

Print "pattern occurs with shift" $i-m$

$v = \pi[v]$ // look for the next match

Compute-Prefix-Function(P) $\Theta(m)$

$m = P \text{ length}$ — 1

Let $\pi[1..m]$ be a new array

$\pi[1] = 0$ — 1

$k = 0$ — 1

for $v = 2$ to m — m

while $k > 0$ and $P[k+1] \neq P[v]$ — $m-1$

$k = \pi[k]$

if $P[k+1] = P[v]$

$k = k + 1$

$\pi[v] = k$

return π

to find π ?

P:	a	b	c	d	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
Index:	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
π	0	0	0	0	1	2	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

P:	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
π	a	b	a	b	a	b	c	a	b	c	a	b	c	a	b	c	a	b	c	a	b	c	a	b	c	a	b	c	a	b
π	0	0	1	2	3	4	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

String Matching Finite Automata

A finite automaton consists of a tuple $(Q, q_0, A, \Sigma, \delta)$, where

- Q is a finite set of states
- $q_0 \in Q$ is the starting state.
- $A \subseteq Q$ is the set of accepting states
- Σ is the input alphabet (finite)
- $\delta: Q \times \Sigma \rightarrow Q$ is the transition functions

State 0 = starting state

Double line boundary = Accepting state

→ This algorithm examines every character exactly once and reports all the valid shifts in $O(n)$.

Complexity $O(n)$
Preprocessing time $O(m|\Sigma|)$

Finite Automaton Matches (T, δ, m)

$n = T$ length

$q = 0$

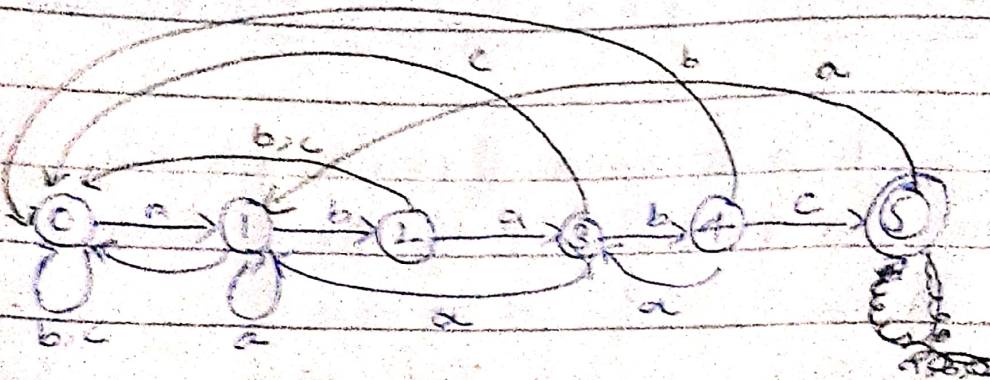
for $i = 1$ to n

$q = \delta(q, T[i])$

if $q == m$

Print "Pattern occurs with shift"

Example: A finite automaton to match pattern $ababcb$ over alphabet $\Sigma = \{a, b, c\}$



State	a	b	c	P
0	1	0	0	a
1	1	2	0	b
2	3	0	0	a
3	1	4	0	b
4	3	0	5	c
5	0	0	0	

$i: 1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8 \ 9 \ 10 \ 11 \ 12 \ 13 \ 14 \ 15 \ 16 \ 17$
 $T[i]: a \ a \ b \ a \ a \ b \ c \ a \ b \ a \ b \ a \ b \ c \ c \ b$
 $q: 0 \ 1 \ 1 \ 2 \ 3 \ 1 \ 2 \ 0 \ 1 \ 2 \ 3 \ 4 \ 3 \ 4 \ 5$
 at shift 10

$i=15, q=5, s=i-q=15-5=10$